Table 1: A comparison of the popular platforms for vibe coding

Platform	Core 'Vibe'/ Persona	Environment type	AI assistance (built-in)	Real-time collaboration	Live preview/ hot reload	Deploy/Hosting integration	Pricing model (Typical)	Offline support	Distinctive strengths	Notable limitations
Replit	Social builder, students, rapid prototypes	Cloud workspace (container/ VM) + browser editor	Ghostwriter (code/chat), AI agents	Yes (multi-player cursors, shared repls)	Instant console and webserver preview	1-click deploy, hosted repl URLs	Free tier + paid plans (usage, AI quota)	Limited (mostly online)	Huge community, template diversity, low friction	Performance limits on free tier; heavier projects need upgrades
GitHub Codespaces	Professional teams needing prod-like dev env	Cloud dev container (VS Code in browser or desktop attach)	GitHub Copilot (integrated)	Live share (optional)	App preview ports with forwarding	GitHub Actions + CI/CD tie-in	Pay-as-you-go compute minutes	Partial (VS Code syncs offline, but codespace is remote)	Environment parity, devcontainer standardisation	Startup latency vs. pure in-browser sandboxes; cost at scale
Cursor	AI-first professional coder seeking deep refactors	Desktop (electron-based) local + remote optional	Multi-agent code generation, repo-aware chat, and inline edits	Limited (file share via Git)	Framework-dependent preview (external)	External (user deploy pipelines)	Freemium + AI usage tiers	Yes (local files)	Strong AI context handling, refactor navigation	Relies on user infra; collaboration basic vs. cloud IDEs
CodeSandbox	Front-end and full-stack quick iteration	Browser + cloud micro-VM/ containers	AI assistant (recent additions)	Yes (shared sessions, forks)	Fast hot reload, storybook and preview panes	Vercel, Netlify, container export	Free + team paid tiers	Mostly online (some local devbox beta)	Fast linkable sandboxes, PR previews	Resource constraints; heavier backends may need external infra
StackBlitz	JS/TS/Node framework devs needing zero install	In-browser WebContainers (no server roundtrip)	Experimental AI (chat/ code)	Yes (collab links)	Instant boot and hot reload	Deploy via integrations (e.g., Vercel)	Free + Pro	Yes (runs local in-browser sandbox)	Near-instant start, fully client-side Node runtime	Limited for non-Web/compiled stacks; backend services constrained
Glitch	Creative front-end/ back-end tinkering and remix culture	Cloud container (always-on toggle)	Basic AI helpers are emerging	Yes (real-time editing)	Auto-reload on save	Built-in hosting (public URLs)	Free + paid boosted apps	Online centric	Low barrier remixing, whimsical social feed	Less suited for complex multi-service architectures
CodePen	Front-end designers, CSS/JS experimentation	Browser (embedded panes)	AI snippet suggestions (growing)	Limited (fork and comment)	Live preview while typing	Export/embed; external deploy needed	Free + Pro (assets, privacy)	Yes (local draft caching)	Visual focus, rapid UI proof-of-concept	Not for back-end; asset limits on free tier
Google Colab	Data scientists, ML experimentation	Cloud notebook (Jupyter variant)	Code completions, model explanations	Yes (commenting, share links)	Cell output; streamlit/gradio via tunnels	Export to Vertex/external; manual deploy	Free + Pro/ Pro+ GPU tiers	Limited offline (notebooks need sync)	Easy GPU access, notebook sharing	Session timeouts; not a full app dev IDE
AWS Cloud9	Cloud-native backend + infra integration	Cloud IDE (EC2/Lambda proximity)	AWS CodeWhisperer	Yes (pair editing)	Preview running apps via ports	Direct AWS deploy (SAM, CDK, CLI)	Pay for underlying compute + minimal IDE charge	No (cloud-based)	Deep AWS IAM and service integration	Less polished UI vs. newer entrants; AWS lock-in
Zed	Fast, minimalist, team collaboration	Native desktop (Rust)	AI (completion/chat)	Real-time presence and voice rooms	External (runs local dev servers)					